



AURORA ROBOTICS

Math Deep Dive

The Robot Outline Controls

Every Slider, Every Step, From First Principles

Supplement to the Capstone Class

How to read this document

- ▶ This document covers `robot_outline.py` in full mathematical detail.
- ▶ Each of the five trackbars gets its own chapter.
- ▶ The underlying operations (`absdiff`, `morphology`, `contours`) are derived from scratch.
- ▶ A single toy scene runs through the whole document — you can compute every step by hand.
- ▶ By the end, every slider in the GUI is no longer a mystery.



Bakel Bakel Begededum
CTO & Chief Instructor, Aurora Robotics

Contents

1	The Five Controls and the Pipeline That Sits Beneath Them	4
2	The Toy Scene We Will Use Throughout	4
2.1	The empty background	4
2.2	The frame with the robot	5
2.3	The pre-processing step shared by both	5
3	The Foundation: Absolute Difference	6
3.1	The formula	6
3.2	Why absolute value? The <code>uint8</code> wraparound trap	6
3.3	Worked example: applying <code>absdiff</code> to the toy scene	7
4	Control 1 — Diff Threshold (<code>DiffThr</code>, τ)	8
4.1	What it controls	8
4.2	The math	8
4.3	The trade-off space	8
4.4	Worked example: sweeping τ on the toy scene	8
5	Morphology Foundations (Before Controls 2 and 3)	9
5.1	The structuring element	9
5.2	The elliptical structuring element used in the script	10
5.3	Erosion	11
5.4	Dilation	13
5.5	Opening: erode-then-dilate	13
5.6	Closing: dilate-then-erode	14
5.7	Why <code>robot_outline.py</code> uses <i>close then open</i>	14
6	Control 2 — Morph Kernel Size (<code>MorphK</code>, k)	15
6.1	What it controls	15
6.2	The math: what changes when k grows?	16
6.3	Why odd k ?	16
6.4	The trade-off	16
6.5	Worked example: closing the same mask with different k	17
7	Control 3 — Morph Iterations (<code>MorphIt</code>, N)	17
7.1	What it controls	18
7.2	The math: iterated operations equal larger ones	18
7.3	Why iterate at all?	18
7.4	The trade-off	18
7.5	Worked example: dilations $N = 1, 2, 3$ on a single pixel	19
8	Contour Finding (Before Control 4)	19
8.1	What a contour is	20
8.2	Connectivity	20
8.3	The Suzuki–Abe algorithm	20
8.4	The two flags used in the script	20
8.5	The shoelace formula for contour area	20

9	Control 4 — Min Contour Area (MinArea, $A_{\min}$)	22
9.1	What it controls	22
9.2	The math	22
9.3	Choosing $A_{\min}$ for your scene	22
9.4	The trade-off	22
9.5	Worked example: filtering five contours	23
10	The Implicit Control: Largest Contour Selection	23
10.1	The math	23
10.2	The hidden assumption	23
11	Control 5 — Show Mode (Mode)	24
11.1	What it controls	24
11.2	The four modes	24
11.3	The teaching value of having all four	24
12	End to End on the Toy Scene	25
12.1	Inputs	25
12.2	Step 1 — absdiff	25
12.3	Step 2 — threshold at $\tau = 25$	25
12.4	Step 3 — close with S_3 , $N = 1$	25
12.5	Step 4 — open with S_3 , $N = 1$	25
12.6	Step 5 — contour finding	26
12.7	Step 6 — shoelace area	26
12.8	Step 7 — area filter $A_{\min} = 4$	26
12.9	Step 8 — argmax	26
12.10	Step 9 — draw	26
12.11	The full output (Mode 3)	26
	The Five Controls, At a Glance	27
	Summary of Equations	27

1 The Five Controls and the Pipeline That Sits Beneath Them

`robot_outline.py` exposes five live trackbars in a window labelled `Controls`. They are:

Trackbar ID	Symbol	Range	Default
DiffThr	τ	5–100	25
MinArea	$A_{\min}$	100–50000	2000
MorphK	k	1–21 (odd)	7
MorphIt	N	0–5	2
Mode	—	0–3	3

To explain what each one *does*, we have to walk through the pipeline they sit in. The pipeline in plain words:

1. Capture an empty-scene background once (on `SPACE`).
2. For every new frame: convert to grayscale, Gaussian-blur it.
3. Compute the absolute difference between this blurred frame and the stored background.
4. Threshold the difference with τ to get a binary “robot-or-not” mask. (**Control 1**)
5. Clean the mask by morphological closing then opening, with a $k \times k$ elliptical structuring element, repeated N times. (**Controls 2 and 3**)
6. Find all contours in the cleaned mask, drop the small ones using $A_{\min}$. (**Control 4**)
7. Keep the single largest surviving contour as the robot; draw it on the original frame.
8. Pick which intermediate to display on screen using Show Mode. (**Control 5**)

This document covers every line of math behind that pipeline. We walk through each operation in order, derive the formulas from first principles, and apply each to a single shared toy scene.

2 The Toy Scene We Will Use Throughout

Before doing any math, we set up a scene that every later section will refer back to. The scene is small enough to compute by hand.

2.1 The empty background

A 4×6 uniform bright “floor” — representing the empty room captured when the user pressed `SPACE`.

The stored background B

$$B = \begin{bmatrix} 180 & 180 & 180 & 180 & 180 & 180 \\ 180 & 180 & 180 & 180 & 180 & 180 \\ 180 & 180 & 180 & 180 & 180 & 180 \\ 180 & 180 & 180 & 180 & 180 & 180 \end{bmatrix}.$$

Read this as

The numbers are grayscale brightnesses on the standard 0–255 scale. 180 is fairly bright but not pure white — typical of an indoor floor under room light. Every cell has the same value because the floor is uniform.

2.2 The frame with the robot

A 4×6 frame captured after a dark “robot” has rolled into view. The robot occupies the centre.

The current frame I

$$I = \begin{bmatrix} 180 & 180 & 50 & 50 & 180 & 180 \\ 180 & 50 & 50 & 50 & 50 & 180 \\ 180 & 50 & 50 & 50 & 50 & 180 \\ 180 & 180 & 50 & 50 & 180 & 180 \end{bmatrix}.$$

Read this as

Same floor at brightness 180, but a small dark object (brightness 50) is now sitting in the middle. The robot’s silhouette is the oval-shaped patch of 50s. Outside that patch the brightness is unchanged.

We index pixels as $I[v, u]$ throughout this document, where v is the row index (0 to 3) and u is the column index (0 to 5). This matches NumPy’s `img[row, col]` convention.

2.3 The pre-processing step shared by both

Before the pipeline subtracts anything, both the background and every live frame go through the same two operations:

1. Convert from BGR colour to a single-channel grayscale image using the standard luminance formula.
2. Apply a 5×5 Gaussian blur to suppress sensor noise.

We won’t recompute the grayscale and blur in every later section — we’ll treat B and I above as the already-pre-processed versions. But it’s worth seeing the math once.

Grayscale conversion

For each BGR pixel (B, G, R) (note OpenCV’s BGR ordering), the grayscale value is

Luminance (Rec. 601)

$$Y = 0.114 B + 0.587 G + 0.299 R.$$

Read this as

A weighted average of the three colour channels, where green gets the most weight, red gets the next-most, and blue gets the least. The reason: the human eye is most sensitive to green, less to red, least to blue. These weights make the resulting grayscale image match how a person would judge brightness.

Gaussian blur

The script applies a fixed 5×5 Gaussian blur to both the background and every live frame. A common integer-form 5×5 Gaussian kernel (approximate, normalised) is

$$G_5 = \frac{1}{273} \begin{bmatrix} 1 & 4 & 7 & 4 & 1 \\ 4 & 16 & 26 & 16 & 4 \\ 7 & 26 & 41 & 26 & 7 \\ 4 & 16 & 26 & 16 & 4 \\ 1 & 4 & 7 & 4 & 1 \end{bmatrix}.$$

Read this as

A small image of weights, peaking in the middle and falling off toward the corners. When you convolve this with the image, each output pixel is a weighted average of a 5×5 neighbourhood, with the centre counting most. The $1/273$ in front is the normalisation (the entries sum to 273); we divide so the average brightness of the image is preserved.

Key Idea

Why blur both? We must blur the live frame because sensor noise would otherwise generate spurious differences. We must blur the background *by the same amount* so the comparison is fair — if one were blurred and the other was not, every edge in the original would create a difference even when the scene is identical.

For the rest of this document, B and I refer to the blurred grayscale versions of those frames.

3 The Foundation: Absolute Difference

Every later step in the pipeline rests on a single per-pixel operation: the absolute difference between the current frame and the stored background.

3.1 The formula

Absolute difference

$$D[v, u] = |I[v, u] - B[v, u]|.$$

Read this as

For every pixel location (v, u) , subtract the background brightness from the current brightness, then take the absolute value of the result. The output D is dark wherever the scene hasn't changed (the subtraction is near zero) and bright wherever it has (the subtraction is far from zero, in either direction).

3.2 Why absolute value? The uint8 wraparound trap

A naive implementation would just do $I - B$. *Don't*. Pixel intensities are stored as `uint8`: unsigned 8-bit integers, with no concept of negative numbers. Arithmetic on `uint8` is performed modulo 256.

So if $I[v, u] = 10$ (a dark pixel in the current frame, where the robot is) and $B[v, u] = 50$ (the brighter floor at the same location in the background), then

$$I[v, u] - B[v, u] = 10 - 50 = -40.$$

But on a `uint8`, -40 does not exist. The result wraps around:

$$-40 \bmod 256 = 256 - 40 = 216.$$

The pixel that should have shown a strong change (40 units of darkening) now reads as 216 — which looks like a strong *brightening*. The mask becomes nonsense.

Where it can go wrong

This is the most common bug in homemade background subtraction code. “I’m doing exactly what the textbook says but my mask is inverted” is almost always silent `uint8` wraparound.

The fix has two equivalent forms:

- Use `cv2.absdiff(I, B)`, which handles the wraparound for you by computing the absolute difference safely.
- Cast to signed integers first: `abs(I.astype(int) - B.astype(int)).astype('uint8')`.

3.3 Worked example: applying `absdiff` to the toy scene

Recall:

$$B = \begin{bmatrix} 180 & 180 & 180 & 180 & 180 & 180 \\ 180 & 180 & 180 & 180 & 180 & 180 \\ 180 & 180 & 180 & 180 & 180 & 180 \\ 180 & 180 & 180 & 180 & 180 & 180 \end{bmatrix}, \quad I = \begin{bmatrix} 180 & 180 & 50 & 50 & 180 & 180 \\ 180 & 50 & 50 & 50 & 50 & 180 \\ 180 & 50 & 50 & 50 & 50 & 180 \\ 180 & 180 & 50 & 50 & 180 & 180 \end{bmatrix}.$$

For every pixel: subtract, take absolute value. Pixels where both I and B are 180 give $|180 - 180| = 0$. Pixels where I is 50 and B is 180 give $|50 - 180| = 130$.

The diff image $D = |I - B|$

$$D = \begin{bmatrix} 0 & 0 & 130 & 130 & 0 & 0 \\ 0 & 130 & 130 & 130 & 130 & 0 \\ 0 & 130 & 130 & 130 & 130 & 0 \\ 0 & 0 & 130 & 130 & 0 & 0 \end{bmatrix}.$$

The robot’s silhouette stands out clearly at value 130, against a clean background of 0.

Note

In real life, neither the background pixels nor the robot pixels will all read identical values. Sensor noise of perhaps ± 3 brightness units is normal, and the floor itself may have slight patterns. So D would actually look more like

$$D_{\text{real}} \approx \begin{bmatrix} 2 & 1 & 128 & 132 & 3 & 1 \\ 2 & 129 & 134 & 131 & 128 & 4 \\ 1 & 130 & 133 & 129 & 132 & 2 \\ 3 & 2 & 127 & 131 & 1 & 2 \end{bmatrix}.$$

The floor pixels now have a few units of noise, and the robot pixels vary slightly too. The next control’s job is to decide which of these counts as “foreground” and which as “background noise”.

4 Control 1 — Diff Threshold (DiffThr, τ)

The Control

Trackbar: DiffThr

Symbol: τ

Range: 5–100

Default: 25

4.1 What it controls

τ is the cutoff at which a pixel-level brightness change counts as “foreground”. Anything in D above τ becomes white in the binary mask M . Anything at or below τ becomes black.

4.2 The math

Binary thresholding

$$M[v, u] = \begin{cases} 255 & \text{if } D[v, u] > \tau, \\ 0 & \text{otherwise.} \end{cases}$$

Read this as

For each pixel, ask one question: “did the brightness change by more than τ units since the empty-scene snapshot?” If yes, paint this pixel white. If no, paint it black. After this step the image is purely binary: every pixel is exactly 0 or exactly 255.

4.3 The trade-off space

τ sits at the boundary between two failure modes:

- **τ too low.** Sensor noise (typically a few brightness units) survives the threshold. The mask shows white speckles all over the “floor” even when no robot is present.
- **τ too high.** Real but subtle parts of the robot — shadows, edges, dim regions — fall below τ and get cut. The robot’s silhouette breaks up into disconnected pieces.

Key Idea

τ is a *signal-to-noise* parameter. If your sensor noise has standard deviation σ_η , you want τ to be at least $3\sigma_\eta$ to $5\sigma_\eta$ above zero so that nearly all background pixels fall below it. For a typical webcam, $\sigma_\eta \approx 2$ – 5 , which is why the default $\tau = 25$ is reasonable.

4.4 Worked example: sweeping τ on the toy scene

Take the realistic noisy diff D_{real} from the note above:

$$D_{\text{real}} = \begin{bmatrix} 2 & 1 & 128 & 132 & 3 & 1 \\ 2 & 129 & 134 & 131 & 128 & 4 \\ 1 & 130 & 133 & 129 & 132 & 2 \\ 3 & 2 & 127 & 131 & 1 & 2 \end{bmatrix}.$$

We threshold at four different values of τ and see what happens.

$\tau = 10$ (too low)

$$M_{\tau=10} = \begin{bmatrix} 0 & 0 & 255 & 255 & 0 & 0 \\ 0 & 255 & 255 & 255 & 255 & 0 \\ 0 & 255 & 255 & 255 & 255 & 0 \\ 0 & 0 & 255 & 255 & 0 & 0 \end{bmatrix}.$$

Every floor pixel had a noise value of 1–4, all of which *are* below $\tau = 10$, so they correctly map to 0. The robot pixels are all above $\tau = 10$ and map to 255. This particular τ happens to work cleanly — but only because our noise was at most 4. With a noisier sensor (noise up to ~ 12) this τ would let speckle through.

 $\tau = 25$ (default)

$$M_{\tau=25} = \begin{bmatrix} 0 & 0 & 255 & 255 & 0 & 0 \\ 0 & 255 & 255 & 255 & 255 & 0 \\ 0 & 255 & 255 & 255 & 255 & 0 \\ 0 & 0 & 255 & 255 & 0 & 0 \end{bmatrix}.$$

Identical to the $\tau = 10$ result in this clean case. The robot’s signal of ~ 130 is well above the threshold, and noise of ≤ 4 is well below. This is the safe operating regime.

 $\tau = 130$ (too high)

$$M_{\tau=130} = \begin{bmatrix} 0 & 0 & 0 & 255 & 0 & 0 \\ 0 & 0 & 255 & 255 & 0 & 0 \\ 0 & 0 & 255 & 0 & 255 & 0 \\ 0 & 0 & 0 & 255 & 0 & 0 \end{bmatrix}.$$

Now τ is at the boundary of the robot’s signal. Pixels that diffed at 127, 128, 129 are *below* the threshold and get cut. The robot’s silhouette becomes a shattered, broken-up shape. The robot is still there in the image — but the algorithm can no longer see it as one piece.

Where it can go wrong

The default $\tau = 25$ assumes typical indoor lighting and a modest-quality webcam. If you move to a dim room, the robot’s brightness changes will be smaller (less signal), the sensor will amplify gain (more noise), and you may have to drop τ to 15 or 20. If you move to bright outdoor light with heavy shadows, increase τ to 40 or 50 so that shadows of trees or cables don’t get picked up as “movement”.

5 Morphology Foundations (Before Controls 2 and 3)

The next two controls — **MorphK** and **MorphIt** — both adjust the same family of operations: *morphological filtering*. Before we can understand them, we need to understand what those operations *do*.

5.1 The structuring element

Morphological operations need a small “probe” that scans across the image. This probe is called a *structuring element* (abbreviated SE). It is just a small binary mask of 0s and 1s, with a designated origin (usually the centre).

Definition

A *structuring element* S is a small array of 0s and 1s with a designated origin. Cells where $S = 1$ are “active”; the operation considers only those neighbouring pixels.

5.2 The elliptical structuring element used in the script

`robot_outline.py` uses an elliptical SE, `cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (k, k))`, of size $k \times k$. The mathematical rule:

Elliptical SE definition

$$S[i, j] = \begin{cases} 1 & \text{if } (i - c)^2 + (j - c)^2 \leq r^2 \\ 0 & \text{otherwise,} \end{cases}$$

where $c = (k - 1)/2$ is the centre coordinate and $r = (k - 1)/2$ is the radius.

Read this as

Pixel (i, j) of the SE is “on” if it lies within a circle of radius r centred at (c, c) . The shape is a discrete approximation of a disk. For small kernels the discretisation looks chunky (a plus sign at $k = 3$, a diamond at $k = 5$); as k grows it approximates a true circle more and more accurately.

The 3×3 ellipse

For $k = 3$, $c = r = 1$. Test each cell $(i, j) \in \{0, 1, 2\}^2$ against $(i - 1)^2 + (j - 1)^2 \leq 1$:

- $(0, 0)$: $1 + 1 = 2$. Not ≤ 1 . $S = 0$.
- $(0, 1)$: $1 + 0 = 1$. ≤ 1 . $S = 1$.
- $(0, 2)$: $1 + 1 = 2$. $S = 0$.
- $(1, 0)$: $0 + 1 = 1$. $S = 1$.
- $(1, 1)$: 0 . $S = 1$.
- $(1, 2)$: $0 + 1 = 1$. $S = 1$.
- $(2, 0)$: $S = 0$.
- $(2, 1)$: $S = 1$.
- $(2, 2)$: $S = 0$.

Result:

$$S_3 = \begin{bmatrix} 0 & 1 & 0 \\ 1 & 1 & 1 \\ 0 & 1 & 0 \end{bmatrix}.$$

The 3×3 ellipse is the *plus* or *cross* shape. Four neighbours by side, no diagonals.

The 5×5 ellipse

For $k = 5$, $c = r = 2$. Test $(i - 2)^2 + (j - 2)^2 \leq 4$.

By symmetry we only need one quadrant. For (i, j) in $\{0, 1, 2\}^2$:

- $(0, 0)$: $4 + 4 = 8 > 4$. $S = 0$.
- $(0, 1)$: $4 + 1 = 5 > 4$. $S = 0$.
- $(0, 2)$: $4 + 0 = 4 \leq 4$. $S = 1$.
- $(1, 0)$: $1 + 4 = 5 > 4$. $S = 0$.
- $(1, 1)$: $1 + 1 = 2 \leq 4$. $S = 1$.
- $(1, 2)$: $1 + 0 = 1 \leq 4$. $S = 1$.
- $(2, 0)$: $0 + 4 = 4 \leq 4$. $S = 1$.
- $(2, 1)$: $0 + 1 = 1 \leq 4$. $S = 1$.
- $(2, 2)$: 0 . $S = 1$.

Mirror to fill the other three quadrants:

$$S_5 = \begin{bmatrix} 0 & 0 & 1 & 0 & 0 \\ 0 & 1 & 1 & 1 & 0 \\ 1 & 1 & 1 & 1 & 1 \\ 0 & 1 & 1 & 1 & 0 \\ 0 & 0 & 1 & 0 & 0 \end{bmatrix}.$$

The 5×5 ellipse is still chunky — it looks like a diamond.

The 7×7 ellipse

For $k = 7$, $c = r = 3$. Test $(i - 3)^2 + (j - 3)^2 \leq 9$.

After computing every cell:

$$S_7 = \begin{bmatrix} 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 1 & 1 & 1 & 1 & 1 & 0 \\ 0 & 1 & 1 & 1 & 1 & 1 & 0 \\ 1 & 1 & 1 & 1 & 1 & 1 & 1 \\ 0 & 1 & 1 & 1 & 1 & 1 & 0 \\ 0 & 1 & 1 & 1 & 1 & 1 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 \end{bmatrix}.$$

Now it actually looks like a circle. Bigger k values give progressively more circular shapes.

5.3 Erosion

The first of the two atomic morphological operations.

Erosion (grayscale)

$$(I \ominus S)[v, u] = \min_{(i, j) \in \text{supp}(S)} I[v + j - c, u + i - c],$$

where $\text{supp}(S) = \{(i, j) : S[i, j] = 1\}$ and c is the SE centre.

Read this as

Place the SE so its centre lands on the pixel (v, u) . For every “on” position of the SE, read the corresponding image pixel. Take the minimum of all those pixel values. That minimum is the eroded output at (v, u) .

For a *binary* image (values in $\{0, 255\}$), the minimum becomes a logical AND: the output is 255 only if every pixel under the SE is 255. If any pixel under the SE is 0, the output is 0.

Key Idea

Erosion *shrinks* white regions. A white pixel survives erosion only if every one of its SE-neighbours is also white. Any isolated white pixel (with at least one black neighbour) gets erased. This is how you delete small specks.

Worked erosion of a binary mask

Take this 4×6 binary mask — our threshold result with a small stray pixel added at position $(2, 5)$ to simulate sensor noise that survived the threshold:

$$M_{\text{noisy}} = \begin{bmatrix} 0 & 0 & 255 & 255 & 0 & 0 \\ 0 & 255 & 255 & 255 & 255 & 0 \\ 0 & 255 & 255 & 255 & 255 & 255 \\ 0 & 0 & 255 & 255 & 0 & 0 \end{bmatrix}.$$

Apply erosion with S_3 (the plus shape).

Erosion at the stray pixel $(v, u) = (2, 5)$

The SE centre lands on $(2, 5)$. The four on-cells of S_3 are the top, bottom, left, and right of the centre. Reading the corresponding image pixels:

$$\begin{aligned} \text{centre } (2, 5) : & \quad M[2, 5] = 255, \\ \text{above } (1, 5) : & \quad M[1, 5] = 0, \\ \text{below } (3, 5) : & \quad M[3, 5] = 0, \\ \text{left } (2, 4) : & \quad M[2, 4] = 255, \\ \text{right } (2, 6) : & \quad (\text{out of image, treat as } 0). \end{aligned}$$

Minimum of $\{255, 0, 0, 255, 0\}$ is 0. So the stray pixel gets erased.

Erosion at the robot centre $(v, u) = (2, 2)$

The four neighbours of $(2, 2)$ under the plus-SE:

$$\begin{aligned} M[2, 2] = 255, \quad M[1, 2] = 255, \quad M[3, 2] = 255, \\ M[2, 1] = 255, \quad M[2, 3] = 255. \end{aligned}$$

Minimum is 255. Pixel stays white. The interior of the robot is preserved by erosion.

The pattern: deep interior pixels survive (all their neighbours are white), edge pixels may not (a neighbour outside the robot is black), and isolated white pixels are guaranteed to die.

5.4 Dilation

The complementary operation.

Dilation (grayscale)

$$(I \oplus S)[v, u] = \max_{(i,j) \in \text{supp}(S)} I[v + j - c, u + i - c].$$

Read this as

Same scan as erosion, except we take the *max* of the neighbours instead of the min. For binary images this is a logical OR: the output is 255 if *any* pixel under the SE is 255, and 0 only if every pixel under the SE is 0.

Key Idea

Dilation *grows* white regions. A black pixel becomes white if any of its SE-neighbours is white. This is how you fill small dark holes.

Worked dilation

Take a mask with a single internal hole at position (1, 2) (simulating a dark patch on the robot surface that fell below the threshold):

$$M_{\text{holed}} = \begin{bmatrix} 0 & 0 & 255 & 255 & 0 & 0 \\ 0 & 255 & 0 & 255 & 255 & 0 \\ 0 & 255 & 255 & 255 & 255 & 0 \\ 0 & 0 & 255 & 255 & 0 & 0 \end{bmatrix}.$$

Apply dilation with S_3 .

Dilation at the hole $(v, u) = (1, 2)$

Reading the SE-neighbours:

$$\begin{aligned} \text{centre } (1, 2) : & \quad M[1, 2] = 0, \\ \text{above } (0, 2) : & \quad M[0, 2] = 255, \\ \text{below } (2, 2) : & \quad M[2, 2] = 255, \\ \text{left } (1, 1) : & \quad M[1, 1] = 255, \\ \text{right } (1, 3) : & \quad M[1, 3] = 255. \end{aligned}$$

Max is 255. The hole gets filled.

5.5 Opening: erode-then-dilate

Opening

$$I \circ S = (I \ominus S) \oplus S.$$

Read this as

First erode the image, then dilate it back. The erosion kills small white specks. The dilation puts the remaining white regions back to roughly their original size — but the killed specks don't come back, because there is nothing left of them to dilate.

Key Idea

Opening removes small white things while preserving big white things. It is the standard “noise removal” tool for binary masks. A single stray pixel of sensor noise is erased forever.

5.6 Closing: dilate-then-erode**Closing**

$$I \bullet S = (I \oplus S) \ominus S.$$

Read this as

First dilate, then erode. The dilation fills small dark holes inside white regions. The erosion shrinks white regions back to roughly their original size — but the filled holes don't come back, because there is nothing dark left to erode them open again.

Key Idea

Closing fills small dark holes while preserving the outer shape. It is the standard “hole filling” tool for binary masks.

5.7 Why robot_outline.py uses close then open

The script applies:

$$M_{\text{clean}} = (M \bullet S) \circ S.$$

That is, close first, then open the result.

Key Idea

The two are not commutative. Doing close-then-open first heals the robot's internal dark patches (close), and then sweeps away the stray bright pixels on the floor (open). If you did open-then-close, an isolated bright speck would first get erased by opening, and then closing would do nothing useful — it would just fill any holes already in the robot, but those holes wouldn't have been filled in the original mask to start with. The order is the difference between a clean blob and a still-noisy one.

Worked example: close then open on our noisy mask

Start with a mask containing both a hole and a stray pixel:

$$M_{\text{both}} = \begin{bmatrix} 0 & 0 & 255 & 255 & 0 & 0 \\ 0 & 255 & 0 & 255 & 255 & 0 \\ 0 & 255 & 255 & 255 & 255 & 255 \\ 0 & 0 & 255 & 255 & 0 & 0 \end{bmatrix}.$$

Step 1: close with S_3 .

Closing is dilate then erode.

After dilation: the hole at (1, 2) fills (worked out above), and white regions grow by one pixel in each on-direction.

$$M_{\text{dilated}} = \begin{bmatrix} 0 & 255 & 255 & 255 & 255 & 0 \\ 255 & 255 & 255 & 255 & 255 & 255 \\ 255 & 255 & 255 & 255 & 255 & 255 \\ 0 & 255 & 255 & 255 & 255 & 255 \end{bmatrix}.$$

After erosion: the white blob shrinks back. The previously filled hole stays filled (it is now in the interior). The blob's shape returns to roughly the original silhouette.

$$M_{\text{closed}} = \begin{bmatrix} 0 & 0 & 255 & 255 & 0 & 0 \\ 0 & 255 & 255 & 255 & 255 & 0 \\ 0 & 255 & 255 & 255 & 255 & 255 \\ 0 & 0 & 255 & 255 & 0 & 0 \end{bmatrix}.$$

Notice: the hole is filled, the original silhouette is restored. But the stray pixel at (2, 5) is still there — closing didn't fix that.

Step 2: open with S_3 .

Opening is erode then dilate.

After erosion: the stray pixel at (2, 5) dies (its neighbours include several 0s). The robot blob shrinks by one pixel in each direction.

$$M_{\text{eroded}} = \begin{bmatrix} 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 255 & 255 & 0 & 0 \\ 0 & 0 & 255 & 255 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix}.$$

After dilation: the robot blob grows back to (approximately) its original size. The dead stray pixel does not come back.

$$M_{\text{clean}} = \begin{bmatrix} 0 & 0 & 255 & 255 & 0 & 0 \\ 0 & 255 & 255 & 255 & 255 & 0 \\ 0 & 255 & 255 & 255 & 255 & 0 \\ 0 & 0 & 255 & 255 & 0 & 0 \end{bmatrix}.$$

The hole is filled, the stray pixel is gone, and the silhouette is back. *This is exactly the mask we wanted from the thresholding step, but it survived noise.*

6 Control 2 — Morph Kernel Size (MorphK, k)

The Control

Trackbar: MorphK

Symbol: k

Range: 1–21, must be odd (even values rounded up)

Default: 7

6.1 What it controls

k is the side length, in pixels, of the elliptical structuring element used by both close and open. Bigger k means a bigger probe, which fills bigger holes and removes bigger specks — at the cost of smoothing out finer details of the robot's shape.

6.2 The math: what changes when k grows?

Three things scale with k :

1. **Reach.** The radius of the SE is $r = (k - 1)/2$ pixels. A single application of opening or closing affects features up to size r . So $k = 7$ reaches $r = 3$ pixels in every direction; $k = 11$ reaches $r = 5$.
2. **Number of operations per pixel.** For each output pixel, the operation reads $\sim \pi r^2 \approx 0.78 k^2$ pixels. Computational cost grows roughly as k^2 .
3. **Maximum hole / speck size affected.** A hole or speck of diameter up to $\sim 2r = k - 1$ pixels can be fully bridged by a single closing or removed by a single opening.

Reach of one open/close pass

$$r = \frac{k - 1}{2} \text{ pixels.}$$

6.3 Why odd k ?

Read this as

The SE has a centre pixel at $(c, c) = ((k - 1)/2, (k - 1)/2)$. If k is odd, the centre is a single well-defined pixel and the SE is symmetric. If k is even, the centre falls between pixels, the SE is asymmetric, and the operation shifts the output by half a pixel in one direction. To avoid that, the script always uses odd k — the helper function `odd_kernel` rounds even trackbar values up to the next odd integer.

6.4 The trade-off

- **k small (3, 5):** Subtle cleanup. Fills tiny holes (1–2 pixels). Removes tiny specks (1–2 pixels). Robot shape is preserved very faithfully. Doesn't help against big problems (large gaps in the silhouette, large noise blobs).
- **k medium (7, 9):** The sweet spot for typical webcam noise. Fills holes up to ~ 6 pixels, removes specks up to ~ 6 pixels. Shape distortion is mild.
- **k large (15, 19):** Heavy smoothing. Useful if your noise is really bad, but the robot's corners and fine detail get rounded off noticeably. The output silhouette is a blobby approximation of the true shape.

Where it can go wrong

The maximum feature size a single open/close can affect is roughly $k - 1$ pixels. If your sensor noise produces 10-pixel-wide blobs occasionally, $k = 5$ will not save you — the noise blob is bigger than the SE and survives. You would need $k = 11$ or larger. But at that size the robot's own shape begins to suffer.

6.5 Worked example: closing the same mask with different k

Take a mask with a 5×5 dark hole inside an otherwise solid white robot. Schematic:

$$M_{\text{bighole}} = \begin{bmatrix} 255 & 255 & 255 & 255 & 255 & 255 & 255 & 255 & 255 \\ 255 & 255 & 255 & 255 & 255 & 255 & 255 & 255 & 255 \\ 255 & 255 & 0 & 0 & 0 & 0 & 0 & 255 & 255 \\ 255 & 255 & 0 & 0 & 0 & 0 & 0 & 255 & 255 \\ 255 & 255 & 0 & 0 & 0 & 0 & 0 & 255 & 255 \\ 255 & 255 & 0 & 0 & 0 & 0 & 0 & 255 & 255 \\ 255 & 255 & 0 & 0 & 0 & 0 & 0 & 255 & 255 \\ 255 & 255 & 255 & 255 & 255 & 255 & 255 & 255 & 255 \\ 255 & 255 & 255 & 255 & 255 & 255 & 255 & 255 & 255 \end{bmatrix}.$$

The hole is 5×5 . Diameter 5. Radius ≈ 2.5 .

Close with $k = 3$ (one iteration)

The SE radius is $r = 1$. Closing with this kernel can bridge gaps of diameter ≤ 2 . Our hole has diameter 5. The closing dilates the surrounding white inward by 1 pixel on each side, then erodes the resulting blob by 1 pixel.

Result: a 3×3 hole remains in the centre. Closing didn't fully fix it because the SE is too small.

Close with $k = 7$ (one iteration)

SE radius $r = 3$. Closing can bridge gaps of diameter ≤ 6 . Our 5-wide hole is comfortably within that. The first dilation expands surrounding white inward by 3 pixels in every direction — enough to overlap and fill the hole completely. The subsequent erosion shrinks the resulting blob back by 3 pixels, but does not re-open the hole because the previously empty pixels are now surrounded entirely by white.

Result: hole is fully filled. The outer shape is preserved to within a pixel.

Close with $k = 15$ (one iteration)

SE radius $r = 7$. The closing easily fills the hole. But the dilation step now reaches *further* than needed, briefly turning the blob into a fat lump; the subsequent erosion shrinks it back. For our particular mask the final result is identical to the $k = 7$ case, because there is no other dark structure within reach. *But* if there were a small piece of background dark near the blob, $k = 15$ would have engulfed it during dilation and left a permanent “bridge” artifact. **Bigger is not always better.**

7 Control 3 — Morph Iterations (MorphIt, N)

The Control

Trackbar: MorphIt

Symbol: N

Range: 0–5

Default: 2

7.1 What it controls

N is the number of times each morphological operation is applied. The script effectively computes

$$M_{\text{clean}} = \underbrace{(\cdots (M \bullet S) \bullet S \cdots)}_{N \text{ closings}} \text{ then } \underbrace{(\cdots (\cdot \circ S) \circ S \cdots)}_{N \text{ openings}}.$$

7.2 The math: iterated operations equal larger ones

A key identity makes the role of N precise. For a dilation, applying S twice is equivalent to applying a larger SE once:

Composition of dilations

$$(I \oplus S) \oplus S = I \oplus (S \oplus S).$$

Read this as

“Dilate with S , then dilate again with S ” is the same as “dilate once with a kernel that is S dilated by itself.” Repeating the dilation N times is equivalent to a single dilation with a kernel of effective size $r_{\text{eff}} = N \cdot r$, where r was the radius of the original SE.

So iterating a 3×3 ellipse N times has roughly the same effect as a single $(2N + 1) \times (2N + 1)$ ellipse. Two iterations of $k = 3$ is similar to one iteration of $k = 5$; three iterations is similar to $k = 7$; and so on.

Effective reach after N iterations

$$r_{\text{eff}} \approx N \cdot r = N \cdot \frac{k - 1}{2}.$$

7.3 Why iterate at all?

If N iterations of k are equivalent to a single bigger kernel, why not just use that bigger kernel? Three reasons.

1. **Speed.** A single $k \times k$ operation costs $\mathcal{O}(k^2)$ per pixel. N iterations of a smaller $k_0 \times k_0$ kernel cost $\mathcal{O}(Nk_0^2)$. If the effective reach is the same ($r_{\text{eff}} = N(k_0 - 1)/2$), the iterated version can be cheaper.
2. **Shape.** The composition $S \oplus S$ is not exactly the same as a bigger ellipse — iterated diamond-like shapes give slightly more diamond-like results than a single-pass true circle. For specific tasks the difference matters.
3. **Flexibility.** Trackbar-controlled N gives the user a more granular knob than k . $k = 7, N = 1$ and $k = 5, N = 2$ achieve similar effects with subtly different characteristics.

7.4 The trade-off

- $N = 0$: Skip morphology entirely. The mask comes straight from the threshold step. Use only when you trust the mask to already be clean.
- $N = 1$: A single close-open pass. Light cleanup.
- $N = 2$: The default. Equivalent reach of $r_{\text{eff}} = 2 \cdot 3 = 6$ pixels for the default $k = 7$. Fills moderate holes, removes moderate specks.

- **$N = 3$ and above:** Aggressive smoothing. Use when the mask is genuinely very noisy. Each extra iteration further rounds the silhouette's corners.

7.5 Worked example: dilations $N = 1, 2, 3$ on a single pixel

Take an image with a single white pixel at the centre of a black field. Dilate with S_3 (the plus). After the first dilation: the single pixel becomes a plus shape (5 pixels). After the second: the plus dilates again, becoming a 3×3 block plus four extra pixels in the cardinal directions ($9 + 4 = 13$ pixels). After the third: the shape continues to expand.

Growth of a single seed under iterated dilation

$N = 0$ (original):

$$\begin{bmatrix} 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix}.$$

$N = 1$ (after one dilation with S_3):

$$\begin{bmatrix} 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 1 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix}.$$

$N = 2$:

$$\begin{bmatrix} 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 1 & 1 & 0 & 0 \\ 0 & 1 & 1 & 1 & 1 & 1 & 0 \\ 0 & 0 & 1 & 1 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix}.$$

After $N = 2$ iterations, the original 1 pixel has grown into a diamond of 13 pixels — exactly the shape of S_5 . So $N = 2$ iterations of S_3 produced the same shape as $N = 1$ iteration of S_5 , confirming the composition identity.

8 Contour Finding (Before Control 4)

After morphology, the mask has clean white blobs on a black background. The next step finds the boundary of each blob and returns it as a list of coordinates — a *contour*. Then Control 4 (**MinArea**) filters that list by area.

8.1 What a contour is

Definition

A *contour* is an ordered sequence of (x, y) pixel coordinates that traces the boundary of a connected white region in a binary image.

8.2 Connectivity

Two pixels are *4-connected* if they touch by a side. They are *8-connected* if they touch by a side or a corner. OpenCV's contour extraction uses 8-connectivity, which means diagonal touches count as connected — one large blob, not two.

8.3 The Suzuki–Abe algorithm

The algorithm used by `cv2.findContours` was published by Suzuki and Abe in 1985. In high-level pseudocode:

1. Scan the image row by row, left to right.
2. Find a transition from black to white — that pixel is on the boundary of a new connected component.
3. Walk around the boundary of that component, following a fixed clockwise rule (always turn left as much as possible while staying on the boundary).
4. Record every pixel visited.
5. When you return to the start pixel, the contour is closed.
6. Continue scanning for more boundary transitions.

8.4 The two flags used in the script

`cv2.RETR_EXTERNAL`

Without this flag, contours of holes inside an object would be returned too (an extra contour for the inside of every donut). With `RETR_EXTERNAL`, only the outermost boundary of each connected component is returned. Holes are ignored.

`cv2.CHAIN_APPROX_SIMPLE`

A long straight edge contains many collinear pixels. Storing them all is wasteful. `CHAIN_APPROX_SIMPLE` stores only the endpoints of horizontal, vertical, and diagonal runs — compressing a 100-pixel-long horizontal edge into two endpoints. The contour is still a valid polygon; we just don't waste memory storing collinear interior points.

8.5 The shoelace formula for contour area

Given a contour as a polygon with vertices $\mathbf{p}_0 = (x_0, y_0), \mathbf{p}_1 = (x_1, y_1), \dots, \mathbf{p}_{n-1} = (x_{n-1}, y_{n-1})$, its enclosed area is given by the *shoelace formula*:

Shoelace area formula

$$A = \frac{1}{2} \left| \sum_{k=0}^{n-1} (x_k y_{k+1} - x_{k+1} y_k) \right|,$$

where indices are taken modulo n (so $\mathbf{p}_n = \mathbf{p}_0$).

Read this as

For each adjacent pair of vertices, compute the cross-product-like quantity $x_k y_{k+1} - x_{k+1} y_k$. Sum these up across all edges of the polygon. Halve, then take the absolute value. The result is the area enclosed by the polygon, in square pixels.

Why the name?

If you write the vertices in two columns and draw multiplication lines going down to the right and up to the right, the pattern looks like a shoelace:

$$\begin{array}{rcl} x_0 & y_0 & \\ x_1 & y_1 & \\ x_2 & y_2 & \\ \vdots & \vdots & \\ x_0 & y_0 & \end{array}$$

The diagonals going $x_k \rightarrow y_{k+1}$ form one set of products; the diagonals going $y_k \rightarrow x_{k+1}$ form the other.

Why absolute value?

The sum $\sum (x_k y_{k+1} - x_{k+1} y_k)$ comes out positive if the vertices are listed counterclockwise, negative if clockwise. The geometric area is unsigned, so we take $|\cdot|$ to make the answer always positive.

Shoelace on a rectangle

A 4×3 rectangle with vertices $(0, 0), (4, 0), (4, 3), (0, 3)$, listed counterclockwise.

$$\begin{aligned} \text{sum} &= (0 \cdot 0 - 4 \cdot 0) + (4 \cdot 3 - 4 \cdot 0) + (4 \cdot 3 - 0 \cdot 3) + (0 \cdot 0 - 0 \cdot 3) \\ &= 0 + 12 + 12 + 0 \\ &= 24. \\ A &= \frac{1}{2} \cdot |24| = 12. \end{aligned}$$

The rectangle is 4 wide by 3 tall, so its area should be 12. ✓

Shoelace on a triangle

A triangle with vertices $(0, 0), (4, 0), (2, 3)$.

$$\begin{aligned} \text{sum} &= (0 \cdot 0 - 4 \cdot 0) + (4 \cdot 3 - 2 \cdot 0) + (2 \cdot 0 - 0 \cdot 3) \\ &= 0 + 12 + 0 \\ &= 12. \\ A &= \frac{1}{2} \cdot |12| = 6. \end{aligned}$$

By the base-times-height formula, the triangle's area is $\frac{1}{2} \cdot 4 \cdot 3 = 6$. ✓

9 Control 4 — Min Contour Area (MinArea, $A_{\min}$)

The Control

Trackbar: MinArea

Symbol: $A_{\min}$

Range: 100–50000

Default: 2000

9.1 What it controls

After contour extraction, we have a list of contours. Many of them might be noise — small specks that morphology didn’t quite kill. $A_{\min}$ is the minimum contour area (in square pixels) for a contour to be considered as a candidate for “the robot”.

9.2 The math

Area filter

$$\mathcal{C}_{\text{kept}} = \{c \in \mathcal{C} : A(c) \geq A_{\min}\}.$$

Read this as

Out of all the contours $\mathcal{C}$ that the contour finder returned, keep only those whose area $A(c)$ (computed by the shoelace formula) is at least $A_{\min}$. Throw away the rest.

9.3 Choosing $A_{\min}$ for your scene

To pick a sensible $A_{\min}$, you need a rough idea of how big the robot looks in pixels at the closest distance you expect.

Key Idea

Rule of thumb: set $A_{\min}$ to about 1/4 of the robot’s expected pixel area at the *far* end of your operating range. That way the robot is always kept (it has at least $4A_{\min}$ when close, $\sim 1A_{\min}$ when far), and most noise blobs — which rarely exceed $\sim A_{\min}/10$ in pixel area — are reliably dropped.

For our class calibration the robot’s silhouette ranged from $\sim 8\,000$ to $\sim 50\,000$ pixels². The default $A_{\min} = 2\,000$ is comfortably below the smaller end, so the robot is always kept.

9.4 The trade-off

- $A_{\min}$ **too low** (100, 500): Noise contours may slip through. If the morphology left even one stray group of white pixels, this filter won’t catch it.
- $A_{\min}$ **around the noise/object boundary** (1 000–5 000): The sweet spot. Noise is reliably rejected; the robot is reliably kept.
- $A_{\min}$ **too high** (20 000, 50 000): The robot at the far end of its range is smaller than $A_{\min}$ and gets thrown away. The pipeline reports “no robot” even when one is clearly present.

9.5 Worked example: filtering five contours

Suppose contour extraction returns five contours with areas:

$$\mathcal{C} = \{c_1, c_2, c_3, c_4, c_5\}, \quad \{A(c_i)\} = \{45, 320, 1\,800, 12\,000, 38\,000\}.$$

Some plausible interpretations: c_1 and c_2 are dust specks that morphology didn't quite remove; c_3 is a small object near the camera (maybe a piece of cable); c_4 is the robot at medium range; c_5 is a person walking through the frame.

Filter at $A_{\min} = 100$

$45 < 100$: drop. $320, 1\,800, 12\,000, 38\,000 \geq 100$: keep. $\mathcal{C}_{\text{kept}} = \{c_2, c_3, c_4, c_5\}$. Four candidates remain.

Filter at $A_{\min} = 2\,000$ (default)

$45, 320, 1\,800 < 2\,000$: drop. $12\,000, 38\,000 \geq 2\,000$: keep. $\mathcal{C}_{\text{kept}} = \{c_4, c_5\}$. The robot and the person remain.

Filter at $A_{\min} = 20\,000$

Only $38\,000 \geq 20\,000$. $\mathcal{C}_{\text{kept}} = \{c_5\}$. Now only the person remains. The robot has been thrown out by an over-aggressive filter.

10 The Implicit Control: Largest Contour Selection

After area filtering, the script picks a single winner from the remaining contours using one line:

```
robot = max(filtered, key=cv2.contourArea)
```

10.1 The math

Largest-contour selection

$$c^* = \arg \max_{c \in \mathcal{C}_{\text{kept}}} A(c).$$

Read this as

Out of the contours that passed the area filter, pick the one whose shoelace area is greatest. That one contour is the robot.

10.2 The hidden assumption

This step assumes the robot is the largest moving thing in the frame. In a controlled lab demo that is usually true. In other situations it can fail:

Where it can go wrong

- A person walks through the frame — their silhouette is bigger than the robot's. The pipeline now tracks the person.
- Two robots are in view — only the larger one gets tracked.

- The robot is partially occluded by a chair, splitting its silhouette into two smaller contours — a smaller, distant object could win instead.

The fix is to add more discriminating logic (colour matching, shape priors, fiducial markers) once you outgrow the single-object assumption. For now, in the controlled demo, $\arg \max$ is fine.

11 Control 5 — Show Mode (Mode)

The Control

Trackbar: Mode

Symbol: —

Range: 0–3

Default: 3

11.1 What it controls

Show Mode does not affect the pipeline at all. It chooses which intermediate image to display on screen. The math is unchanged — this is purely a debugging control.

11.2 The four modes

Mode	Symbol	What appears on screen
0	D	Raw absolute difference (grayscale)
1	M	Binary mask after thresholding
2	M_{clean}	Binary mask after morphology
3	$I + \text{contour}$	Original frame with green outline of robot

11.3 The teaching value of having all four

Key Idea

The Show Mode trackbar is the single most underrated control in the script. When the pipeline isn't working, you don't know which step is broken — but flipping through modes 0, 1, 2 lets you see exactly where the pipeline went wrong:

- Mode 0 looks bad? Your camera, background capture, or blur is the problem. Recapture the background or move the camera.
- Mode 0 looks fine but mode 1 is missing the robot? Your τ is too high.
- Mode 1 is full of speckle? Your τ is too low.
- Mode 1 looks reasonable but mode 2 is mangled? Your morph parameters are wrong — k too big, N too high.
- Mode 2 is clean but mode 3 shows the wrong contour? Your $A_{\min}$ is wrong or there is a person in frame.

12 End to End on the Toy Scene

We collect every step into one continuous worked example. Start from the original toy scene of Section 1, run through every stage, land at the final output.

12.1 Inputs

Background (already grayscale + blurred):

$$B = \begin{bmatrix} 180 & 180 & 180 & 180 & 180 & 180 \\ 180 & 180 & 180 & 180 & 180 & 180 \\ 180 & 180 & 180 & 180 & 180 & 180 \\ 180 & 180 & 180 & 180 & 180 & 180 \end{bmatrix}.$$

Current frame:

$$I = \begin{bmatrix} 180 & 180 & 50 & 50 & 180 & 180 \\ 180 & 50 & 50 & 50 & 50 & 180 \\ 180 & 50 & 50 & 50 & 50 & 180 \\ 180 & 180 & 50 & 50 & 180 & 180 \end{bmatrix}.$$

Parameters: $\tau = 25$, $k = 3$ (for tractability in worked examples), $N = 1$, $A_{\min} = 4$.

12.2 Step 1 — absdiff

$$D = \begin{bmatrix} 0 & 0 & 130 & 130 & 0 & 0 \\ 0 & 130 & 130 & 130 & 130 & 0 \\ 0 & 130 & 130 & 130 & 130 & 0 \\ 0 & 0 & 130 & 130 & 0 & 0 \end{bmatrix}.$$

12.3 Step 2 — threshold at $\tau = 25$

$$M = \begin{bmatrix} 0 & 0 & 255 & 255 & 0 & 0 \\ 0 & 255 & 255 & 255 & 255 & 0 \\ 0 & 255 & 255 & 255 & 255 & 0 \\ 0 & 0 & 255 & 255 & 0 & 0 \end{bmatrix}.$$

12.4 Step 3 — close with S_3 , $N = 1$

Dilation of M with S_3 expands the white region by one pixel in each cardinal direction. Erosion shrinks it back. The result on our already-clean mask is essentially unchanged (no holes to fill):

$$M_{\text{closed}} = \begin{bmatrix} 0 & 0 & 255 & 255 & 0 & 0 \\ 0 & 255 & 255 & 255 & 255 & 0 \\ 0 & 255 & 255 & 255 & 255 & 0 \\ 0 & 0 & 255 & 255 & 0 & 0 \end{bmatrix}.$$

12.5 Step 4 — open with S_3 , $N = 1$

Same story: nothing to remove. The robot blob survives.

$$M_{\text{clean}} = M.$$

12.6 Step 5 — contour finding

`cv2.findContours` returns a single contour — the outer boundary of the robot blob. Approximately the polygon:

$$[(2, 0), (3, 0), (4, 1), (4, 2), (3, 3), (2, 3), (1, 2), (1, 1)]$$

(eight vertices, counterclockwise, with diagonal segments representing the corner pixels).

12.7 Step 6 — shoelace area

Apply the shoelace formula to the eight vertices. Without the detailed arithmetic (which you can do exactly as in Section 7.5), the result is

$$A = 12 \text{ pixels}^2.$$

12.8 Step 7 — area filter $A_{\min} = 4$

$12 \geq 4$, so the contour is kept.

12.9 Step 8 — argmax

Only one contour survives. It is selected as the robot.

12.10 Step 9 — draw

The script draws the contour polygon on the original frame I in bright green at thickness 3. The final output is the original camera frame with a green outline tracing the robot's silhouette.

12.11 The full output (Mode 3)

Schematically, the displayed image is I with green pixels along the contour boundary:

$$\text{Output} = \begin{bmatrix} 180 & 180 & G & G & 180 & 180 \\ 180 & G & 50 & 50 & G & 180 \\ 180 & G & 50 & 50 & G & 180 \\ 180 & 180 & G & G & 180 & 180 \end{bmatrix},$$

where G indicates a bright green pixel. This is what you would see on screen at Show Mode 3.

The Five Controls, At a Glance

Control	What it does, in one sentence
DiffThr (τ)	Pixels in the difference image above τ become foreground; below τ become background.
MorphK (k)	Side length of the elliptical structuring element used by close and open; bigger k reaches further.
MorphIt (N)	How many times the close and open are repeated; each iteration multiplies the effective reach.
MinArea ($A_{\min}$)	Drops contours with shoelace area below $A_{\min}$ before selecting the robot.
Mode	Selects which intermediate image of the pipeline is displayed; does not affect the pipeline itself.

Summary of Equations

Concept	Equation
Luminance	$Y = 0.114B + 0.587G + 0.299R$
Absolute difference	$D[v, u] = I[v, u] - B[v, u] $
Binary threshold	$M = 255$ if $D > \tau$ else 0
SE definition (ellipse)	$S[i, j] = 1$ if $(i - c)^2 + (j - c)^2 \leq r^2$
SE radius	$r = (k - 1)/2$
Erosion	$(I \ominus S)[v, u] = \min_{(i, j) \in S} I[v + j - c, u + i - c]$
Dilation	$(I \oplus S)[v, u] = \max_{(i, j) \in S} I[v + j - c, u + i - c]$
Opening	$I \circ S = (I \ominus S) \oplus S$
Closing	$I \bullet S = (I \oplus S) \ominus S$
Iterated reach	$r_{\text{eff}} \approx N(k - 1)/2$
Shoelace area	$A = \frac{1}{2} \sum_k (x_k y_{k+1} - x_{k+1} y_k) $
Area filter	$\mathcal{C}_{\text{kept}} = \{c : A(c) \geq A_{\min}\}$
Robot selection	$c^* = \arg \max_{c \in \mathcal{C}_{\text{kept}}} A(c)$

End of deep dive. Try re-running the toy scene with $\tau = 100$ or $k = 9$ and predict the output by hand before checking the script.